

Отчётик

1. Общее описание проекта

Платформа реализована как fullstack-приложение на Next.js 15 (App Router) с использованием Prisma ORM 5.22, SQLite (в dev-режиме) и TypeScript. Архитектура подразумевает разделение пользователей на три роли: user, developer, admin, каждая из которых имеет изолированный функционал и доступ к различным модулям системы.

Технологический стек

Компонент	Версия / Технология
Framework	Next.js v15.0.0
Language	TypeScript v5.5.0
ORM	Prisma v5.22.0
Database	SQLite (via prisma/client)
Auth	iron-session v8.0.1
Password hashing	bcryptjs v2.4.3
Styling	TailwindCSS v3.4.0
Runtime	ts-node v10.9.2

2. Архитектура базы данных (Prisma Schema)

2.1. Центральные сущности и их связи

User — ядро системы. Реализована **полносвязная модель** с 15+ отношениями: - **Контент**: articles, newsPosts, comments, reviews, games (для разработчиков) - **Социальные функции**: followers / following (через таблицу Follow с @unique([followerId, followingId])), conversations1/2 (двусторонняя связь для чата) - **Системная**: tickets (созданные), assignedTickets (назначенные — для саппорта/админов), subscriptions, notifications, sentMessages - **Ролевая модель**: поле role со значениями user | developer | admin, isActive

```
model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  name        String
  passwordHash String
  role        String       @default("user") // user, developer, admin
  createdAt   DateTime     @default(now())
  isActive    Boolean      @default(true)
  tickets     Ticket[]     @relation("author")
  assignedTickets Ticket[] @relation("assignee")
  articles    Article[]
  newsPosts   NewsPost[]
  subscriptions Subscription[]
  comments    Comment[]
  reviews     GameReview[]
  notifications Notification[]
  avatarUrl   String?
  games       Game[]
  genres      Genre[]
  lastSeenAt  DateTime?
  followers   Follow[] @relation("UserFollowing") // те, кто подписан на меня (я following)
  following   Follow[] @relation("UserFollowers")  // те, на кого я подписан (я follower)
  age         Int?
  showFollowers Boolean @default(false)
  conversations1 Conversation[] @relation("User1Conversations")
  conversations2 Conversation[] @relation("User2Conversations")
  sentMessages Message[] @relation("SentMessages")
}
```

Game — центральная сущность платформы: - Связана с User (author), Ticket[], Article[], NewsPost[], SubscriptionTier[], GameReview[], GameMedia[], GameGenre[], GamePlatform[], GameFile[] - Поле requiredTier реализует **доступ по уровню подписки** (null — открыто, “bronze”/“silver”/“gold” — платный контент)

```
model Game {
  id          Int          @id @default(autoincrement())
  title       String
  description  String?
  mediaUrl    String?
  authorId    Int?
  author      User?        @relation(fields: [authorId], references: [id])
  createdAt   DateTime     @default(now())
  tickets     Ticket[]
  articles    Article[]
  newsPosts   NewsPost[]
  subscriptionTiers SubscriptionTier[]
  reviews     GameReview[]
  media        GameMedia[]
  gameGenres  GameGenre[]
  gameFileUrl String?      // URL к APK/EXE
  gamePlatforms GamePlatform[]
  gameFiles    GameFile[]
  requiredTier String?     // null (открыто), "bronze", "silver", "gold"
}
```

2.2. Система подписок (монетизация)

Реализована двухуровневая модель: - **SubscriptionTier** — шаблон подписки (название, цена Float, описание преимуществ, maxSlots), привязан к игре (gameId) - **Subscription** — конкретная активная подписка пользователя (startDate, endDate, статус: active | cancelled | expired

```
model SubscriptionTier {
  id          Int          @id @default(autoincrement())
  name        String
  price       Float
  benefits    String
  maxSlots    Int          @default(100)
  gameId      Int
  game        Game         @relation(fields: [gameId], references: [id], onDelete: Cascade)
  createdAt   DateTime     @default(now())
  subscriptions Subscription[]
}

model Subscription {
  id          Int          @id @default(autoincrement())
  startDate   DateTime     @default(now())
  endDate     DateTime?
  status      String       @default("active") // active, cancelled, expired
  userId      Int
  user        User         @relation(fields: [userId], references: [id])
  tierId       Int
  tier         SubscriptionTier @relation(fields: [tierId], references: [id], onDelete: Cascade)
}
```

2.3. Система тикетов (техподдержка)

Модель **Ticket** с полным жизненным циклом: - Типизация: type (bug/feature/question), priority (low/medium/high), status (open/in_progress/resolved/closed) - Назначение: assignee (связь с User) — позволяет реализовать распределение задач между саппортом - Резолюция: resolution (для resolved), closeReason (для closed) — аудит закрытия

```
model Ticket {
  id          Int          @id @default(autoincrement())
  title       String
  description  String
  type        String       @default("bug") // bug, feature, question
  priority    String       @default("medium") // low, medium, high
  status      String       @default("open") // open, in_progress, resolved, closed
  createdAt   DateTime     @default(now())
  gameId      Int
  game        Game         @relation(fields: [gameId], references: [id], onDelete: Cascade)
  authorId    Int
  author      User         @relation("author", fields: [authorId], references: [id])
  assigneeId  Int?
  assignee    User?        @relation("assignee", fields: [assigneeId], references: [id])
  resolution  String?      // описание решения (для статуса resolved)
  closeReason String?      // причина закрытия (для статуса closed)
}
```

2.4. Рекурсивные комментарии

Модель **Comment** с самореферентной связью:

parentId Int?

parent Comment? @relation("CommentReplies", fields: [parentId],
references: [id], onDelete: Cascade)

replies Comment[] @relation("CommentReplies")

Это позволяет строить **дерево вложенных ответов** (threads) к
статьям.

```
model Comment {  
  id      Int      @id @default(autoincrement())  
  content String  
  createdAt DateTime @default(now())  
  articleId Int?  
  article Article? @relation(fields: [articleId], references: [id], onDelete: Cascade)  
  authorId Int  
  author  User      @relation(fields: [authorId], references: [id], onDelete: Cascade)  
  parentId Int?  
  parent  Comment? @relation("CommentReplies", fields: [parentId], references: [id], onDelete: Cascade)  
  replies Comment[] @relation("CommentReplies")  
}
```

2.5. Уникальные ограничения и целостность данных

GameReview: @unique([gameId, authorId]) — гарантия **одного**
отзыва на игру от пользователя

```
model GameReview {  
  id      Int      @id @default(autoincrement())  
  rating  Int       // 1-5  
  content String?  
  createdAt DateTime @default(now())  
  updatedAt DateTime @updatedAt  
  
  gameId Int  
  game   Game      @relation(fields: [gameId], references: [id], onDelete: Cascade)  
  
  authorId Int  
  author  User      @relation(fields: [authorId], references: [id], onDelete: Cascade)  
  
  @@unique([gameId, authorId]) // один отзыв от пользователя на игру  
}
```

Follow: `@unique([followerId, followingId])` — предотвращение дублирования подписок

```
model Follow {
  id          Int          @id @default(autoincrement())
  followerId  Int
  follower    User          @relation("UserFollowers", fields: [followerId], references: [id], onDelete: Cascade)
  followingId Int
  following    User          @relation("UserFollowing", fields: [followingId], references: [id], onDelete: Cascade)
  createdAt   DateTime @default(now())

  @@unique([followerId, followingId])
}
```

Conversation: `@unique([user1Id, user2Id])` — единый чат между двумя пользователями

```
model Conversation {
  id          Int          @id @default(autoincrement())
  user1Id     Int
  user1       User          @relation("User1Conversations", fields: [user1Id], references: [id], onDelete: Cascade)
  user2Id     Int
  user2       User          @relation("User2Conversations", fields: [user2Id], references: [id], onDelete: Cascade)
  lastMessageAt DateTime @default(now())
  messages    Message[]

  clearedByUser1 Boolean @default(false)
  clearedByUser2 Boolean @default(false)
  deletedByUser1 Boolean @default(false)
  deletedByUser2 Boolean @default(false)

  @@unique([user1Id, user2Id])
}
```

GameGenre, GamePlatform: составные первичные ключи
`@id([gameId, genreId])` — корректная many-to-many реализация

```
model GameGenre {
  gameId Int
  game    Game @relation(fields: [gameId], references: [id], onDelete: Cascade)
  genreId Int
  genre    Genre @relation(fields: [genreId], references: [id], onDelete: Cascade)

  @@id([gameId, genreId])
}
```

```
model GamePlatform {
  gameId      Int
  game         Game @relation(fields: [gameId], references: [id], onDelete: Cascade)
  platformId  Int
  platform     Platform @relation(fields: [platformId], references: [id], onDelete: Cascade)

  @@id([gameId, platformId])
}
```

2.6. Каскадное удаление и стратегии onDelete

Продуманная политика удаления: - Cascade — для зависимого контента (медиа, отзывы, тикеты игры, сообщения чата) - SetNull — для необязательных связей (статья может потерять привязку к игре при её удалении)

3. Архитектура приложения (Next.js App Router)

3.1. Структура роутинга

Используется **App Router** (src/app/) с чётким разделением по доменам:

Роут	Назначение
/admin	Административная панель
/developer/dashboard, /developer/tickets	Личный кабинет разработчика
/games/[id]	Карточка игры (edit, news, subscribe)
/guides, /knowledge	База знаний / статьи
/messages	Личные сообщения
/profile, /user/[id]	Профили пользователей
/api/...	REST API (более 30 эндпоинтов)

3.2. API-слой (src/app/api/)

Видна **глубокая иерархия REST API**: - **Auth**: login/, logout/, register/, me/, change-password/ - **Content**: articles/, news/, comments/, games/, genres/, platforms/ - **Game-specific**: games/[id]/files/, media/, reviews/ — загрузка файлов, медиа, отзывы - **Business-logic**: subscribe/, subscription-tiers/, tickets/, my-tickets/, notifications/ - **Social**: user/follow/, followers/, profile/, users/search/, messages/[id]/conversation/ - **Upload**: upload/route.ts — отдельный эндпоинт для загрузки файлов

3.3. Компонентная архитектура (src/components/)

Выделены **предметные компоненты**, отражающие бизнес-логику: - **CRUD-формы**: CreateGameForm, CreateArticleForm, CreateNewsForm, CreateTicketForm, EditGameButton, DeleteGameButton - **Медиа**: CoverUploader, MediaUploader, GameGallery - **Подписки**: SubscribeButton, TierSelect, DeveloperSubscriptions - **Социальное**: FollowButton, CommentSection, GameCommentSection, GameReviewsSection, SendMessageButton, ExistingChatButton, GroupChat - **Тикеты**: TicketSection, DeveloperTickets, QuickStats - **Фильтры/поиск**: GenreSelect, GenreMultiSelect, PlatformSelect

4. Выделенные сложные и ключевые части системы

4.1. Middleware и передача pathname (src/middleware.ts)

Почему важно: Middleware в Next.js 15 App Router — единственная точка входа для обработки всех HTTP-запросов до попадания в роуты. Здесь реализована инъекция заголовка x-pathname, который используется клиентскими компонентами для определения текущего пути.

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const response = NextResponse.next();
  response.headers.set('x-pathname', request.nextUrl.pathname);
  return response;
}
```

Сложность: В отличие от Page Router, где useRouter() давал pathname на клиенте, в App Router серверные компоненты не имеют прямого доступа к request-контексту без middleware. Это решение позволяет layout-компонентам и навигации синхронизироваться с текущим URL.

4.2. Сессионная аутентификация через iron-session

(src/lib/session.ts)

Почему важно: Используется **stateless cookie-based** аутентификация через iron-session — современную альтернативу JWT с шифрованием на стороне сервера. Cookie помечаются как secure только в production.

```
export const sessionOptions: SessionOptions = {
  password: process.env.SECRET_COOKIE_PASSWORD || 'complex_password_at_least_32_characters_long',
  cookieName: 'game-support-session',
  cookieOptions: {
    secure: process.env.NODE_ENV === 'production',
  },
};

export async function getSession() {
  const cookieStore = await cookies();
  const session = await getIronSession<SessionData>(cookieStore, sessionOptions);
  return session;
}
```

Сложность: iron-session шифрует данные сессии в cookie с использованием Seal (iron), что избавляет от необходимости хранить сессии в БД или Redis. Однако требует строгого управления cookieName и password через переменные окружения.

4.3. Ролевая модель и авторизация (src/lib/roles.ts)

Почему важно: Централизованное определение ролей с human-readable метками. Роль хранится в сессии и проверяется на каждом защищённом API-эндпоинте.

```
const roleLabels: Record<string, string> = {  
  user: 'Пользователь',  
  developer: 'Разработчик',  
  admin: 'Администратор',  
};  
  
export function getRoleLabel(role: string): string {  
  return roleLabels[role] || role;  
}
```

Сложность: Проверка ролей в API (например, `session.role !== 'developer' && session.role !== 'admin'`) дублируется в десятках роутов. Это потенциальная точка для внедрения middleware-авторизации или HOF (Higher-Order Function) для защиты роутов.

4.4. Система подписок с проверкой дубликатов (src/app/api/subscribe/route.ts)

Почему важно: Ядро монетизации. POST-метод реализует **идемпотентную логику подписки** — проверяет, нет ли уже активной подписки на эту игру, прежде чем создать новую.

```
export async function POST(request: Request) {
  const session = await getSession();
  if (!session.userId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const { tierId } = await request.json();
  const tier = await prisma.subscriptionTier.findUnique({ where: { id: tierId } });
  if (!tier) return NextResponse.json({ error: 'Tier not found' }, { status: 404 });
}
```

// Проверяем, нет ли уже активной подписки на эту игру

```
const existing = await prisma.subscription.findFirst({
  where: {
    userId: session.userId,
    status: 'active',
    tier: { gameId: tier.gameId },
  },
});
if (existing) {
  return NextResponse.json({ error: 'Already subscribed to this game' }, { status: 400 });
}

const subscription = await prisma.subscription.create({
  data: {
    userId: session.userId,
    tierId: tier.id,
    status: 'active',
    startDate: new Date(),
  },
  include: { tier: { include: { game: true } } },
});

return NextResponse.json(subscription, { status: 201 });
}
```

Сложность: Запрос `findFirst` с вложенным условием `tier: { gameId: ... }` требует понимания реляционных фильтров Prisma. Вложенный `include` возвращает полную цепочку `subscription → tier → game` за один запрос, что оптимизирует N+1 проблему.

4.5. Статусная машина тикетов с уведомлениями
(src/app/api/tickets/[id]/route.ts)

Почему важно: Реализована полноценная workflow-логика обработки тикетов с валидацией переходов, обязательными полями и системой уведомлений.

```
const validStatuses = ['open', 'in_progress', 'resolved', 'closed'];
if (!status || !validStatuses.includes(status)) {
  return NextResponse.json({ error: 'Invalid status' }, { status: 400 });
}
```

// Для закрытия обязательно указать причину

```
if (status === 'closed' && !closeReason?.trim()) {
  return NextResponse.json({ error: 'Причина закрытия обязательна' }, { status: 400 });
}
```

// Если переводим в решён — сохраняем описание

```
if (status === 'resolved') {
  data.resolution = resolution?.trim() || null;
}
```

// При возврате из закрытого/решиённого — сбрасываем поля

```
if (status !== 'resolved' && status !== 'closed') {
  data.resolution = null;
  data.closeReason = null;
}

const updatedTicket = await prisma.ticket.update({
  where: { id: ticketId },
  data,
  include: { author: true },
});
```

// Уведомление автору

```
if (updatedTicket.authorId !== session.userId) {
  const statusText = statusLabels[status] || status;
  await prisma.notification.create({
    data: {
      userId: updatedTicket.authorId,
      type: 'ticket_status',
      message: `Статус вашего тикета «${updatedTicket.title}» изменён на «${statusText}»`,
      relatedId: updatedTicket.id,
    },
  });
}
```

Сложность: 1. Валидация бизнес-правил на уровне API (не только БД) 2. Условная логика очистки полей при смене статуса 3. **Side-effect** — создание уведомления в той же транзакции логики (хотя без \$transaction есть риск рассогласования) 4. **Проверка прав** — только developer и admin могут менять статус

4.6. Фильтрация тикетов разработчика по его играм

(src/app/api/developer/tickets/route.ts)

Почему важно: Демонстрирует многоэтапную агрегацию данных — сначала получаем игры разработчика, затем тикеты только по этим играм.

```
export async function GET() {
  const session = await getSession();
  if (!session.userId || (session.role !== 'developer' && session.role !== 'admin')) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const developerGames = await prisma.game.findMany({
    where: { authorId: session.userId },
    select: { id: true },
  });
  const gameIds = developerGames.map(g => g.id);

  const tickets = await prisma.ticket.findMany({
    where: { gameId: { in: gameIds } },
    include: {
      game: { select: { title: true } },
      author: { select: { name: true } },
    },
    orderBy: { createdAt: 'desc' },
  });

  return NextResponse.json(tickets);
}
```

Сложность: Использование `where: { in: gameIds }` — массивный фильтр по внешнему ключу. В production на больших объёмах требует индекса `gameId + createdAt`.

4.7. Компонент подписки с динамическим UI

(src/components/SubscribeButton.tsx)

Почему важно: Клиентский компонент реализует **сложный UI-стейт** — загрузку tier-ов, проверку активной подписки, переключение между состояниями.

```
export default function SubscribeButton({
  gameId,
  authorId,
}: {
  gameId: number;
  authorId?: number | null;
}) {
  const [user, setUser] = useState<any>(null);
  const [tiers, setTiers] = useState<any[]>([]);
  const [subscription, setSubscription] = useState<any>(null);
  const [showTiers, setShowTiers] = useState(false);
  const [message, setMessage] = useState('');

  useEffect(() => {
    fetch('/api/auth/me')
      .then(res => res.json())
      .then(data => setUser(data.user));

    fetch(`/api/subscription-tiers?gameId=${gameId}`)
      .then(res => res.json())
      .then(data => {
        if (Array.isArray(data)) {
          setTiers(data);
        } else {
          setTiers([]);
        }
      })
      .catch(() => setTiers([]));
  }, [gameId]);

  useEffect(() => {
    if (!user) return;
    fetch('/api/subscribe')
      .then(res => res.json())
      .then(data => {
        if (Array.isArray(data)) {
          const activeSub = data.find(
            (s: any) => s.tier?.gameId === gameId && s.status === 'active'
          );
          setSubscription(activeSub || null);
        }
      });
  }, [gameId, user]);
```

```

const handleSubscribe = async (tierId: number) => {
  if (!user) {
    alert('Необходимо войти');
    return;
  }
  const res = await fetch('/api/subscribe', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ tierId }),
  });
  if (res.ok) {
    // Перезагружаем страницу, чтобы сервер узнал о новой подписке
    window.location.reload();
  } else {
    const err = await res.json();
    setMessage(err.error || 'Ошибка');
  }
};

const handleCancel = async () => {
  if (!subscription) return;
  await fetch('/api/subscribe', {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ subscriptionId: subscription.id }),
  });
  // Перезагружаем страницу после отмены
  window.location.reload();
};

```

Сложность: 1. Каскадные `useEffect` с зависимостями — сначала `auth`, потом `tiers`, потом `subscription` 2. Клиент-side фильтрация активной подписки по `gameId` среди всех подписок пользователя 3. Оптимистичный UI — после успешного запроса идёт полная перезагрузка страницы для синхронизации серверного состояния

4.8. Рекурсивные комментарии с визуальной иерархией

(src/components/CommentSection.tsx)

Почему важно: Реализована **рекурсивная визуализация дерева комментариев** с ограничением глубины, сворачиванием и ответами.

```
function CommentThread({
  comment,
  allComments,
  onReply,
  depth = 0,
}): {
  comment: Comment;
  allComments: Comment[];
  onReply: (parentId: number) => void;
  depth: number;
}) {
  const [collapsed, setCollapsed] = useState(true);

  const childComments = allComments.filter(c => c.parentId === comment.id);
  const hasReplies = childComments.length > 0;

  const preview =
    comment.content.length > 100
      ? comment.content.slice(0, 100) + '...'
      : comment.content;
```

```
if (collapsed) {
  return (
    <div
      style={{
        marginLeft: depth * 20,
        marginTop: '0.5rem',
        borderLeft: depth > 0 ? '2px solid #d1d5db' : 'none',
        paddingLeft: depth > 0 ? '1rem' : 0,
      }}
    >
      <div className="flex items-center gap-2">
        <Link href={`/${user}/${comment.author.id}`} className="font-medium text-sm text-gray-900 dark:text-gray-100">
          {comment.author.name}
        </Link>
        <span className="text-gray-500 dark:text-gray-400 text-xs">{preview}</span>
        <button onClick={() => setCollapsed(false)} className="text-xs text-indigo-600 dark:text-indigo-400 hover:underline">
          Показать
        </button>
      </div>
    </div>
  );
}
```

```

<p className="text-sm text-gray-800 dark:text-gray-200 mb-1">{comment.content}</p>
<div className="flex gap-2">
  <button onClick={() => onReply(comment.id)} className="text-xs text-indigo-600 dark:text-indigo-400 hover:underline">
    Ответить
  </button>
  <button onClick={() => setCollapsed(true)} className="text-xs text-gray-500 dark:text-gray-400 hover:underline">
    Скрыть
  </button>
</div>

```

Сложность: 1. **Рекурсивный рендеринг** в React — компонент вызывает сам себя для child-элементов 2. **Визуальная индентация** через `marginLeft` и `borderLeft`, зависящие от `depth` 3. **Оптимизация** — `collapsed state` предотвращает рендеринг всего дерева сразу 4. **Фильтрация корневых комментариев** — `rootComments = comments.filter(c => c.parentId === null)`

4.9. Чат с мягким удалением и уведомлениями
(`src/app/api/messages/conversation/[id]/route.ts`)

Почему важно: Реализована **сложная логика приватности** — мягкое удаление диалога (`deletedByUser1/2`), очистка истории (`clear`) и физическое удаление только при согласии обоих участников.

```

export async function PATCH(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {

```

```
const session = await getSession();
```

```
const { id } = await params;
const conversationId = Number(id);
```

```
const conversation = await prisma.conversation.findUnique({
  where: { id: conversationId },
  select: { user1Id: true, user2Id: true, deletedByUser1: true, deletedByUser2: true },
});
```

```
const isUser1 = session.userId === conversation.user1Id;
const isUser2 = session.userId === conversation.user2Id;
```

```
const body = await request.json();
const { action } = body;

if (action === 'delete') {
```

// Если диалог с самим собой — удаляем физически

```
if (conversation.user1Id === conversation.user2Id) {
  await prisma.conversation.delete({ where: { id: conversationId } });
  return NextResponse.json({ deleted: true, allDeleted: true });
}
```

// Устанавливаем флаг удаления для текущего пользователя

```
if (isUser1) {
  await prisma.conversation.update({
    where: { id: conversationId },
    data: { deletedByUser1: true },
  });
} else {
  await prisma.conversation.update({
    where: { id: conversationId },
    data: { deletedByUser2: true },
  });
}
```

// Если оба пользователя удалили — физическое удаление

```
const updated = await prisma.conversation.findUnique({
  where: { id: conversationId },
  select: { deletedByUser1: true, deletedByUser2: true },
});

if (updated?.deletedByUser1 && updated?.deletedByUser2) {
  await prisma.conversation.delete({ where: { id: conversationId } });
  return NextResponse.json({ deleted: true, allDeleted: true });
}

return NextResponse.json({ deleted: true });
}

if (action === 'clear') {
  await prisma.message.deleteMany({ where: { conversationId } });
  return NextResponse.json({ cleared: true });
}

return NextResponse.json({ error: 'Неизвестное действие' }, { status: 400 });
}
```

Сложность: 1. **Мягкое удаление** — данные сохраняются для второго участника диалога 2. **Кооперативное удаление** — физическое удаление только при `deletedByUser1 && deletedByUser2` 3. **Edge-case** — диалог “с самим собой” удаляется сразу 4. **Разделение действий** — `delete` (скрыть для себя) vs `clear` (удалить сообщения)

4.10. Загрузка файлов с валидацией (src/app/api/upload/route.ts)

Почему важно: Единый эндпоинт для загрузки медиа-контента с проверкой типа, размера и генерацией уникальных имён.

```

import { NextResponse } from 'next/server';
import { writeFile, mkdir } from 'fs/promises';
import path from 'path';

export async function POST(request: Request) {
  try {
    const formData = await request.formData();
    const file = formData.get('file') as File | null;

    if (!file) {
      return NextResponse.json({ error: 'Файл не выбран' }, { status: 400 });
    }

    const allowedTypes = ['image/jpeg', 'image/png', 'image/gif', 'image/webp', 'video/mp4', 'video/webm'];
    if (!allowedTypes.includes(file.type)) {
      return NextResponse.json({ error: 'Неподдерживаемый тип файла' }, { status: 400 });
    }

    const maxSize = 10 * 1024 * 1024; // 10 МБ
    if (file.size > maxSize) {
      return NextResponse.json({ error: 'Файл слишком большой (макс. 10 МБ)' }, { status: 400 });
    }

    const timestamp = Date.now();
    const ext = file.name.split('.').pop() || 'bin';
    const fileName = `${timestamp}-${Math.random().toString(36).substring(2)}.${ext}`;

    const uploadDir = path.join(process.cwd(), 'public', 'uploads');
    await mkdir(uploadDir, { recursive: true });

    const buffer = Buffer.from(await file.arrayBuffer());
    await writeFile(path.join(uploadDir, fileName), buffer);

    return NextResponse.json({ url: `/uploads/${fileName}` });
  } catch (error: any) {
    console.error('Ошибка загрузки файла:', error.message);
    return NextResponse.json(
      { error: 'Внутренняя ошибка сервера при загрузке файла' },
      { status: 500 }
    );
  }
}

```

Сложность: 1. Валидация на уровне API — тип MIME + размер файла до записи на диск 2. Безопасное именование — timestamp + random предотвращает коллизии и path traversal 3. **Stream vs Buffer** — используется arrayBuffer() → Buffer, что для 10 МБ приемлемо, но для больших файлов требует streaming 4. **Синхронизация с БД** — URL возвращается клиенту, который затем сохраняет его в GameMedia / ArticleMedia

4.11. Список диалогов с непрочитанными сообщениями (src/app/api/messages/route.ts)

Почему важно: GET-метод реализует агрегацию данных для списка чатов — последнее сообщение, количество непрочитанных, информация о собеседнике.

```
export async function GET() {
  const session = await getSession();
  if (!session.userId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const conversations = await prisma.conversation.findMany({
    where: {
      OR: [
        { user1Id: session.userId },
        { user2Id: session.userId },
      ],
    },
    include: {
      user1: { select: { id: true, name: true, avatarUrl: true } },
      user2: { select: { id: true, name: true, avatarUrl: true } },
      messages: {
        orderBy: { createdAt: 'desc' },
        take: 1,
      },
      _count: {
        select: { messages: true },
      },
    },
    orderBy: { lastMessageAt: 'desc' },
  });
}
```

```

// Для каждого диалога считаем непрочитанные
const result = await Promise.all(
  conversations.map(async (conv) => {
    const partner = conv.userId === session.userId ? conv.user2 : conv.user1;
    const lastMessage = conv.messages[0] || null;

    const unreadCount = await prisma.message.count({
      where: {
        conversationId: conv.id,
        isRead: false,
        senderId: { not: session.userId },
      },
    });

    return {
      id: conv.id,
      partner,
      lastMessage: lastMessage
        ? {
            text: lastMessage.text,
            createdAt: lastMessage.createdAt,
            isMine: lastMessage.senderId === session.userId,
          }
        : null,
      unreadCount,
    };
  })
);

return NextResponse.json(result);
}

```

Сложность: 1. **N+1 проблема** — Promise.all с map + message.count() для каждого диалога. Для сотен диалогов требует оптимизации через groupBy 2. **Денормализация** — поле lastMessageAt в Conversation обновляется при каждом новом сообщении для быстрой сортировки 3. **Полиморфный ответ** — структура result собирается вручную, не соответствует прямому типу Prisma

5. Заключение

Проект демонстрирует **продвинутое владение современным стеком Next.js**: - App Router с серверными и клиентскими компонентами - Prisma ORM со сложными реляциями, транзакциями и агрегациями - Stateless аутентификация через iron-session - Ролевая модель с защитой API - Монетизация через tier-based подписки - Рекурсивные структуры данных (комментарии) - Мягкое удаление и приватность в чате - Загрузка и валидация медиа-файлов

Ключевые сложности сосредоточены в **бизнес-логике подписок, workflow(рабочего процесса) тикетов, рекурсивной обработке комментариев и многосторонней логике чата** — эти модули требуют глубокого понимания как Prisma, так и архитектурных паттернов Next.js.

1. Workflow - рабочий процесс

2. Рекурсивная(ый) - ситуация, когда объект или процесс включает в себя самого себя в описании или действии

3. Денормализация - намеренное приведение структуры базы данных в состояние, не соответствующее критериям нормализации, с целью повышения производительности операций чтения за счёт добавления избыточных данных

4. Полиморфный - принцип, который позволяет объектам разных классов обрабатывать вызовы через единый интерфейс, адаптируя своё поведение в зависимости от конкретного типа. Один и тот же метод может реализовываться по-разному в зависимости от типа объекта.

5. Агрегация - отношение между классами, которое описывает связь типа «часть-целое» или «содержит»

6. Идентация - способ оформления программного кода с помощью пробелов или табуляции в начале строк для визуального выделения структурных блоков кода. Цель — сделать структуру кода более очевидной, облегчить чтение и понимание его логики, а также избежать путаницы при вложенности блоков.

7. UI-состояние (UI state) - это данные, которые описывают текущее состояние пользовательского интерфейса (UI) в приложении. Оно включает информацию, необходимую для отрисовки элементов интерфейса, их отображения и реакции на действия пользователя(в данном контексте имеется ввиду про **Состояние элементов UI** - свойства, которые влияют на то, как элемент отображается. Например, элемент может быть показан или скрыт, иметь определённый шрифт, размер или цвет.).